thinking in filesystem paths, then `parent/child` comes quite naturally). In the same way, `parent::` becomes `..` again, just like a filesystem path. The full list of axis abbreviations is as follows:

Table 11-12. *XPath axis abbreviations*

Long form	Abbreviation	
`child::`	blank	`/child::div/child::p/` becomes `/div/p`
`attribute::`	`@`	`node/attribute::id` becomes `node/@id`
`descendant-or-self::`	`//`	`/descendant-or-self::node()/child::thing` becomes `//thing`
`parent::`	`..`	`//parent::main` becomes `//main/..`
`self::`	`.`	`[self::node()//*contains(...)]` becomes `[.//*contains(...)]`

The last example is a sneak peek of predicate syntax. (A note to the cautious: the last item `[.//*contains()]` does not return the same set as `[//*contains()]`, and you should always rigorously test your expressions – if stuff looks weird, try adding a `self::` axis).

Node Tests

A node test is usually very simple: a match on the node (tag) name. No special syntax is required: you can use a localization path of `axis::node_name`, and the XPath engine knows that you meant "find me all the nodes of this name on this axis from the current context." I also mentioned that the node test component can be against node type, and for this, you need a function (yes, XPath has functions, 25 of them, in fact, in the XPath 1.0 spec):